Portfolio – Phase 2

Projekt: Data-Mart-Erstellung in SQL

(DLBDSPBDM01_D)

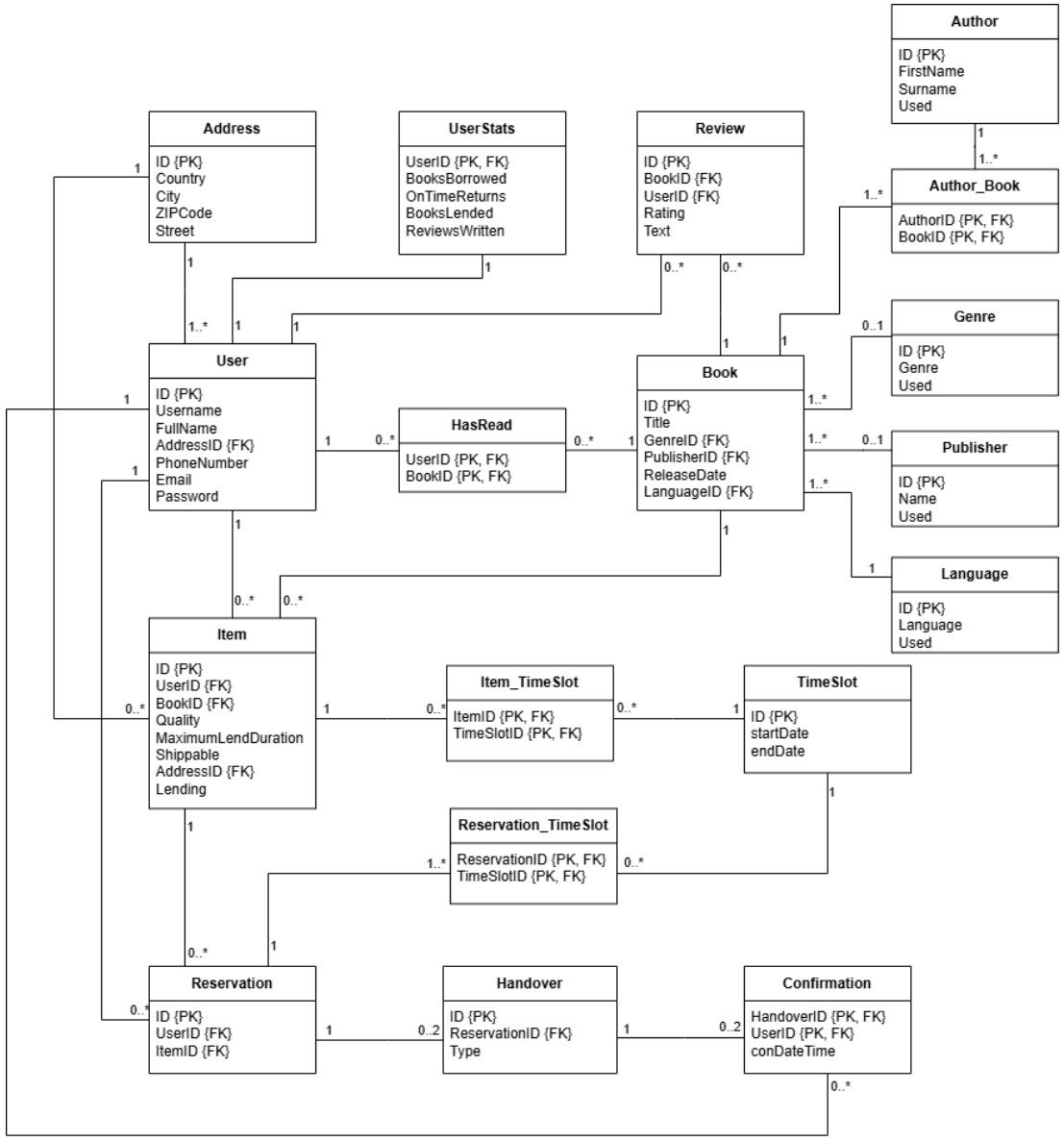
Verfasser, Mat.Nr.: Lukas Baumgarten, IU14137995

Tutorin: Anna Androvitsanea

Inhalt

- Entity-Relationship-Modell
- Address
- User
- Publisher
- Genre
- Author
- Language
- Book
- Author_Book
- HasRead
- Review
- Item
- TimeSlot
- Item_TimeSlot
- Reservation
- Reservation_TimeSlot
- Handover
- Confirmation
- UserStats
- Zusammenfassung

Entity-Relationship-Modell



ERM Buchtausch-App, eigene Darstellung

Address

- **Erstellen der Tabelle**

```
CREATE TABLE Address (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  Country VARCHAR(50) NOT NULL,
  City VARCHAR(50) NOT NULL,
  ZIPCode VARCHAR(10) NOT NULL,
  Street VARCHAR(100) NOT NULL);
```

- **Erstellen von Trigger, um unreferenzierte Entitäten zu löschen**

```
[…] AFTER UPDATE OR DELETE ON User FOR EACH ROW
[…] IF ((SELECT COUNT(*) FROM User WHERE AddressID = OLD.AddressID) = 0) THEN
    DELETE FROM Address WHERE ID = OLD.AddressID; […]
```

- **Testfall: Füge Testdaten ein, lösche und ändere diese, um zu prüfen, ob die referenzierten Address Entitäten gelöscht werden**

```
[…] CREATE OR REPLACE PROCEDURE test_address()
[…]  DECLARE EXIT HANDLER FOR SQLEXCEPTION, SQLWARNING
    ROLLBACK;
  START TRANSACTION;
    INSERT INTO Address VALUES […];
    INSERT INTO User VALUES […];
    DELETE FROM User WHERE Username = 'Test A';
    UPDATE User SET AddressID = 10 WHERE Username = 'Test B'; […]
  SELECT * FROM Address WHERE NOT ID IN (SELECT AddressID FROM User);
  DELETE FROM User WHERE ID = LAST_INSERT_ID()+1; […]
CALL test_address();
```

Screenshot MariaDB: Dummy-Daten in Address

```
MariaDB [bookmart]> SELECT * FROM Address;
+----+-------------+-----------+---------+---------------------------------+
| ID | Country     | City      | ZIPCode | Street                          |
+----+-------------+-----------+---------+---------------------------------+
|  1 | Deutschland | Berlin    | 10779   | Barbarossastraße 24             |
|  2 | Deutschland | Berlin    | 10779   | Barbarossastraße 26             |
|  3 | Deutschland | Berlin    | 10779   | Münchener Str. 45               |
|  4 | Deutschland | Berlin    | 10779   | Münchener Str. 11               |
|  5 | Deutschland | Berlin    | 10825   | Münchener Str. 25 a             |
|  6 | Deutschland | Berlin    | 10825   | Münchener Str. 25 b             |
|  7 | Deutschland | Potsdam   | 14482   | Turnstraße 28                   |
|  8 | Deutschland | Potsdam   | 14482   | Turnstraße 29                   |
|  9 | Deutschland | Magdeburg | 39128   | Grünewaldweg 21                 |
| 10 | Polen       | Szczecin  | 71-157  | Księdza Grzegorza Piramowicza 5 |
+----+-------------+-----------+---------+---------------------------------+
10 rows in set (0.000 sec)
```

Erwartetes Testergebnis:
Empty set

Tatsächliches Testergebnis:

Screenshot MariaDB: Testergebnis Address

```
MariaDB [bookmart]> CALL test_address();
Empty set (0.004 sec)

Query OK, 9 rows affected (0.005 sec)
```

User

- **Erstellen der Tabelle**

```
CREATE TABLE User (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  Username VARCHAR(30) NOT NULL,
  FullName VARCHAR(100) NOT NULL,
  AddressID INT UNSIGNED NOT NULL,
  PhoneNumber VARCHAR(30),
  Email VARCHAR(50),
  Password BLOB(32) NOT NULL,
  FOREIGN KEY (AddressID) REFERENCES Address (ID)
);
```

Screenshot MariaDB: Dummy-Daten in User

```
MariaDB [bookmart]> SELECT * FROM User ORDER BY Username ASC LIMIT 3;
+----+--------------+--------------+-----------+-----------------+--------------------------+-------
| ID | Username     | FullName     | AddressID | PhoneNumber     | Email                    | Passwo
+----+--------------+--------------+-----------+-----------------+--------------------------+-------
|  2 | anna.schmidt | Anna Schmidt |         1 | +49 170 1234501 | anna.schmidt@example.com | b2f6e9
| 10 | emma.schulz  | Emma Schulz  |         9 | +49 178 1234509 | emma.schulz@example.com  | 42c20e
|  7 | felix.becker | Felix Becker |         6 | +49 175 1234506 | felix.becker@example.com | 0dd64c
+----+--------------+--------------+-----------+-----------------+--------------------------+-------
3 rows in set (0.001 sec)
```

- **Testfall User: Prüfe, ob das Löschen einer verwendeten Address ordnungsgemäß Fehlschlägt**

Erwartetes Testergebnis:
Foreign Key Error

Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall User

```
MariaDB [bookmart]> DELETE FROM Address WHERE ID = (SELECT AddressID FROM User LIMIT 1);
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`bookmart`.`user`, CONSTRAINT `1` FOREIGN KEY (`AddressID`) REFERENCES `addres
s` (`ID`))
```

Publisher

- **Erstellen der Tabelle**

```
CREATE TABLE Publisher (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(500) NOT NULL,
  Used BOOL DEFAULT 1);
```

- **Erstelle Trigger, welcher Language, Genre und Publisher Entitäten markiert, wenn sie nicht referenziert werden**

```
[…] CREATE TRIGGER BookDeleted_language_genre_publisher
AFTER UPDATE OR DELETE ON Book FOR EACH ROW
  IF ((SELECT COUNT(*) FROM Book WHERE LanguageID = OLD.LanguageID) = 0) THEN
    UPDATE Language SET Used = 0 WHERE ID = OLD.LanguageID;
  END IF;
  IF ((SELECT COUNT(*) FROM Book WHERE GenreID = OLD.GenreID) = 0) THEN
    UPDATE Genre SET Used = 0 WHERE ID = OLD.GenreID;
  END IF;
  IF ((SELECT COUNT(*) FROM Book WHERE PublisherID = OLD.PublisherID) = 0) THEN
    UPDATE Publisher SET Used = 0 WHERE ID = OLD.PublisherID;
[…]
```

- **Testfall Publisher: Siehe Testfall Language**

Screenshot MariaDB: Dummy-Daten in Publisher

```
MariaDB [bookmart]> SELECT * FROM Publisher ORDER BY ID ASC LIMIT 3;
+----+-----------------+------+
| ID | Name            | Used |
+----+-----------------+------+
|  1 | Fischer Verlag  |    1 |
|  2 | Rowohlt Verlag  |    1 |
|  3 | Suhrkamp Verlag |    1 |
+----+-----------------+------+
3 rows in set (0.000 sec)
```

Genre

- **Erstellen der Tabelle**

```
CREATE TABLE Genre (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  Genre VARCHAR(200) NOT NULL,
  Used BOOL DEFAULT 1);
```

- **Erstellter Trigger: Siehe Publisher**
- **Testfall Publisher: Siehe Testfall Language**

Screenshot MariaDB: Dummy-Daten in Genre

```
MariaDB [bookmart]> SELECT * FROM Genre ORDER BY ID ASC LIMIT 3;
+----+-----------------+------+
| ID | Genre           | Used |
+----+-----------------+------+
|  1 | Roman           |    1 |
|  2 | Fantasy         |    1 |
|  3 | Science-Fiction |    1 |
+----+-----------------+------+
3 rows in set (0.001 sec)
```

Language

- **Erstellen der Tabelle**

```
CREATE TABLE Language (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, Language VARCHAR(200) NOT NULL, Used BOOL DEFAULT 1);
```

- **Erstellter Trigger: Siehe Publisher**
- **Testfall Publisher, Genre, Language: Prüfe, ob unreferenzierte Entitäten ordungsgemäß markiert werden**

```
[…] CREATE OR REPLACE PROCEDURE test_langPubGen()
[…] DECLARE langIDa, langIDb, genIDa, genIDb, pubIDa, pubIDb TYPE OF Language.ID;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION, SQLWARNING ROLLBACK; […]
    INSERT INTO Language VALUES (NULL, 'Bsp. Language A', 1), (NULL, 'Bsp. Language B', 1);
    SET langIDa = LAST_INSERT_ID(); […]
    INSERT INTO Genre VALUES (NULL, 'Bsp. Genre A', 1), (NULL, 'Bsp. Genre B', 1);
    SET genIDa = LAST_INSERT_ID(); […]
    INSERT INTO Publisher VALUES (NULL, 'Bsp. Publisher A', 1), (NULL, 'Bsp. Publisher B', 1);
    SET pubIDa = LAST_INSERT_ID(); […]
    INSERT INTO Book VALUES […]
    UPDATE Book SET GenreID = 1, PublisherID = 1, LanguageID = 1 WHERE ID = LAST_INSERT_ID();
    DELETE FROM Book WHERE ID = LAST_INSERT_ID()+1; COMMIT;
  SELECT * FROM Publisher WHERE ID NOT IN (SELECT PublisherID FROM Book);
  SELECT * FROM Genre WHERE ID NOT IN (SELECT GenreID FROM Book);
  SELECT * FROM Language WHERE ID NOT IN (SELECT LanguageID FROM Book);
  DELETE FROM Book […] DELETE FROM Publisher […] DELETE FROM Genre […] DELETE FROM Language [...]
CALL test_langPubGen();
```

Screenshot MariaDB: Dummy-Daten in Publisher

```
MariaDB [bookmart]> SELECT * FROM Language;
+----+----------+------+
| ID | Language | Used |
+----+----------+------+
|  1 | Deutsch  |    1 |
|  2 | English  |    1 |
+----+----------+------+
2 rows in set (0.001 sec)
```

Erwartetes Testergebnis:

Jeweils „Bsp. [..] A“ und B der drei Tabellen mit Used = 0

Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall Language, Publisher und Genre

```
MariaDB [bookmart]> CALL test_langPubGen();
+----+------------------+------+
| ID | Name             | Used |
+----+------------------+------+
| 36 | Bsp. Publisher A |    0 |
| 37 | Bsp. Publisher B |    0 |
+----+------------------+------+
2 rows in set (0.004 sec)

+----+-------------+------+
| ID | Genre       | Used |
+----+-------------+------+
| 35 | Bsp. Genre A |    0 |
| 36 | Bsp. Genre B |    0 |
+----+-------------+------+
2 rows in set (0.005 sec)

+----+----------------+------+
| ID | Language       | Used |
+----+----------------+------+
| 19 | Bsp. Language A |    0 |
| 20 | Bsp. Language B |    0 |
+----+----------------+------+
2 rows in set (0.006 sec)

Query OK, 23 rows affected (0.008 sec)
```

Book

- **Erstellen der Tabelle**

```
CREATE TABLE Book (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  Title VARCHAR(500) NOT NULL,
  GenreID INT UNSIGNED,
  PublisherID INT UNSIGNED,
  ReleaseDate DATE,
  LanguageID INT UNSIGNED NOT NULL,
  FOREIGN KEY (GenreID) REFERENCES Genre (ID),
  FOREIGN KEY (PublisherID) REFERENCES Publisher (ID),
  FOREIGN KEY (LanguageID) REFERENCES Language (ID));
```

- **Testfall: Prüfe, ob die Tabellen Korrekt Verknüpft sind**

Screenshot MariaDB: Dummy-Daten in Book

```
MariaDB [Bookmart]> SELECT * FROM Book ORDER BY Title ASC LIMIT 3;
+----+------------------+---------+-------------+-------------+------------+
| ID | Title            | GenreID | PublisherID | ReleaseDate | LanguageID |
+----+------------------+---------+-------------+-------------+------------+
|  5 | Cold Case        |       5 |           7 | 2021-09-09  |          1 |
|  4 | Der letzte Zeuge |       4 |           6 | 2019-05-15  |          1 |
|  1 | Der stille Wald  |       1 |           1 | 2015-03-12  |          1 |
+----+------------------+---------+-------------+-------------+------------+
3 rows in set (0.001 sec)
```

Erwartetes Testergebnis:
Cold Case | Thriller | Diogenes Verlag | Deutsch
Der letzte Zeuge | Krimi | Piper Verlag | Deutsch
Der stille Wald | Roman | Fischer Verlag | Deutsch
Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall Book

```
MariaDB [Bookmart]> SELECT Title, Genre.Genre, Publisher.Name, Language.Language FROM Book LEFT JOIN Genre ON Book.GenreID = Genre.ID LEFT JOIN Publisher ON Book.PublisherID = Publ
isher.ID LEFT JOIN Language ON Book.LanguageID = Language.ID ORDER BY Title ASC LIMIT 3;
+------------------+----------+-----------------+----------+
| Title            | Genre    | Name            | Language |
+------------------+----------+-----------------+----------+
| Cold Case        | Thriller | Diogenes Verlag | Deutsch  |
| Der letzte Zeuge | Krimi    | Piper Verlag    | Deutsch  |
| Der stille Wald  | Roman    | Fischer Verlag  | Deutsch  |
+------------------+----------+-----------------+----------+
3 rows in set (0.004 sec)
```

Author

- **Erstellen der Tabelle**

```
CREATE TABLE Author (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  FirstName VARCHAR(50),
  Surname VARCHAR(100) NOT NULL,
  Used BOOL DEFAULT 1);
```

- **Trigger: Markiere unreferenzierte Einträge in Author**

```
[…] CREATE OR REPLACE TRIGGER markUnused_author
AFTER UPDATE OR DELETE ON Author_Book FOR EACH ROW
IF ((SELECT COUNT(*) FROM Author_Book WHERE AuthorID = OLD.AuthorID) = 0) THEN
    UPDATE Author SET Used = 0 WHERE ID = OLD.AuthorID; […]
```

- **Testfall: Prüfe, ob unreferenzierte Einträge ordnungsgemäß markiert werden**

Prozedur für Testfall: Siehe Author_Book

Screenshot MariaDB: Dummy-Daten in Author

```
MariaDB [bookmart]> SELECT * FROM Author ORDER BY ID ASC LIMIT 3;
+----+-----------+----------+------+
| ID | FirstName | Surname  | Used |
+----+-----------+----------+------+
|  1 | Thomas    | Bergmann |    1 |
|  2 | Julia     | Neumann  |    1 |
|  3 | Michael   | Krüger   |    1 |
+----+-----------+----------+------+
3 rows in set (0.000 sec)
```

Erwartetes Testergebnis:
[…] | Author A | 0
[…] | Author B | 0
Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall Author, Author_Book

```
MariaDB [bookmart]> CALL test_authorBook();
+----+-----------+----------+------+
| ID | FirstName | Surname  | Used |
+----+-----------+----------+------+
| 60 | NULL      | Author A |    0 |
| 61 | NULL      | Author B |    0 |
+----+-----------+----------+------+
2 rows in set (0.009 sec)

+----------+--------+
| AuthorID | BookID |
+----------+--------+
|        1 |     92 |
+----------+--------+
1 row in set (0.010 sec)

Query OK, 16 rows affected (0.011 sec)
```

Author_Book

- **Erstellen der Tabelle**

```
CREATE TABLE Author_Book (
  AuthorID INT UNSIGNED,
  BookID INT UNSIGNED,
  PRIMARY KEY (AuthorID, BookID),
  FOREIGN KEY (AuthorID) REFERENCES Author (ID),
  FOREIGN KEY (BookID) REFERENCES Book (ID)
    ON UPDATE CASCADE);
```

- **Trigger: ON DELETE CASCADE Replikation**

```
CREATE OR REPLACE TRIGGER BookDeleted_AuthorBook
BEFORE DELETE ON Book FOR EACH ROW
  DELETE FROM Author_Book WHERE BookID = OLD.ID;
```

- **Testfall: Prüfe, ob Änderungen aus Book übernommen werden**

```
[…] CREATE OR REPLACE PROCEDURE test_authorBook() […]
DECLARE bID, aID TYPE OF Book.ID;
  INSERT INTO Book VALUES (NULL, 'Book A', NULL, NULL, NULL, 1), […]; SET bID =
LAST_INSERT_ID();
  INSERT INTO Author VALUES (NULL, NULL, 'Author A'), […];  SET aID =
LAST_INSERT_ID();
  INSERT INTO Author_Book VALUES (aID, bID), (aID+1, bID+1);
  UPDATE Book SET ID = bID+2 WHERE ID = bID; UPDATE Author_Book SET AuthorID = 1
WHERE BookID = bID+2;
  DELETE FROM Book WHERE ID = bID+1;
  SELECT * FROM Author WHERE ID NOT IN (SELECT AuthorID FROM Author_Book);
  SELECT * FROM Author_Book WHERE BookID IN (bID+1, bID+2);
[…] CALL test_authorBook();
```

Screenshot MariaDB: Dummy-Daten in Author_Book

```
MariaDB [bookmart]> SELECT * FROM Author_Book ORDER BY BookID DESC LIMIT 3;
+----------+--------+
| AuthorID | BookID |
+----------+--------+
|       10 |     10 |
|       11 |      9 |
|        9 |      9 |
+----------+--------+
3 rows in set (0.000 sec)
```

Erwartetes Testergebnis:
1 | [BookID]
Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall Author, Author_Book

```
MariaDB [bookmart]> CALL test_authorBook();
Empty set (0.009 sec)

+----------+--------+
| AuthorID | BookID |
+----------+--------+
|        1 |     38 |
+----------+--------+
1 row in set (0.009 sec)

Query OK, 14 rows affected (0.010 sec)
```

HasRead

- **Erstellen der Tabelle**

```
CREATE OR REPLACE TABLE HasRead (
  UserID INT UNSIGNED,
  BookID INT UNSIGNED,
  PRIMARY KEY (UserID, BookID),
  FOREIGN KEY (UserID) REFERENCES User (ID)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  FOREIGN KEY (BookID) REFERENCES Book (ID)
    ON UPDATE CASCADE
    ON DELETE CASCADE);
```

Screenshot MariaDB: Dummy-Daten in HasRead

```
MariaDB [bookmart]> SELECT * FROM HasRead ORDER BY UserID ASC LIMIT 3;
+--------+--------+
| UserID | BookID |
+--------+--------+
|      2 |      2 |
|      2 |      3 |
|      2 |      4 |
+--------+--------+
3 rows in set (0.000 sec)
```

- **Testfall: Prüfe die korrekte Verknüpfung der Tabellen**

Erwartetes Testergebnis:
User.Username | Book.Title (Übereinstimmend mit manueller Überprüfung)
Tatsächliches Testergebnis:
(Korrekt)

Screenshot MariaDB: Testfall HasRead

```
MariaDB [bookmart]> SELECT User.Username, Book.Title FROM HasRead LEFT JOIN User ON User.ID = HasRead.UserID LEFT JOIN Book ON Book.ID = HasRead.BookID LIMIT 3;
+---------------+--------------+
| Username      | Title        |
+---------------+--------------+
| anna.schmidt  | Drachenfeuer |
| jonas.fischer | Drachenfeuer |
| sophie.wagner | Drachenfeuer |
+---------------+--------------+
3 rows in set (0.000 sec)
```

Review

- **Erstellen der Tabelle**

```
CREATE OR REPLACE TABLE Review (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  BookID INT UNSIGNED,
  UserId INT UNSIGNED,
  Rating TINYINT UNSIGNED NOT NULL,
  Text TEXT,
  FOREIGN KEY (BookID) REFERENCES Book (ID),
  FOREIGN KEY (UserID) REFERENCES User (ID),
  CONSTRAINT ratingRange CHECK (Rating BETWEEN 0 AND 10),
  UNIQUE INDEX (BookID, UserID));
```

Screenshot MariaDB: Dummy-Daten in Review

```
MariaDB [bookmart]> SELECT * FROM Review ORDER BY LENGTH(Text) ASC LIMIT 3;
+----+--------+--------+--------+-----------------------------------------------------------------------+
| ID | BookID | UserId | Rating | Text                                                                  |
+----+--------+--------+--------+-----------------------------------------------------------------------+
|  5 |      5 |      9 |     10 | Super!!                                                               |
|  8 |      8 |      4 |      5 | Interessante Ansätze, aber streckenweise zu trocken geschrieben.      |
|  9 |      9 |      6 |     10 | Meine Kinder lieben dieses Buch! Wird bei uns jeden Abend vorgelesen. |
+----+--------+--------+--------+-----------------------------------------------------------------------+
3 rows in set (0.000 sec)
```

- **Testfall: Zeige alle Rezensionen zu einem Buch**

Erwartetes Testergebnis:
max.mueller | Der stille Wald | 8 | (Review Text)
Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall Review

```
MariaDB [bookmart]> SELECT User.Username, Book.Title, Rating, Text FROM Review LEFT JOIN User ON User.ID = Review.UserID LEFT JOIN Book ON Book.ID = Review.BookID WHERE BookID = 1;
+-------------+-----------------+--------+-----------------------------------------------------------------------------------------------+
| Username    | Title           | Rating | Text                                                                                          |
+-------------+-----------------+--------+-----------------------------------------------------------------------------------------------+
| max.mueller | Der stille Wald |      8 | Ein wunderschön geschriebenes Buch, das mich von der ersten bis zur letzten Seite gefesselt hat.  |
+-------------+-----------------+--------+-----------------------------------------------------------------------------------------------+
1 row in set (0.001 sec)
```

Item

- **Erstellen der Tabelle**

```
CREATE OR REPLACE TABLE Item (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  UserID INT UNSIGNED NOT NULL, BookID INT UNSIGNED NOT NULL,
  Quality VARCHAR(200),
  MaximumLendDuration INT UNSIGNED DEFAULT 30,
  Shippable BOOL, AddressID INT UNSIGNED NOT NULL, Lending ENUM('disabled', 'restricted', 'free') NOT NULL,
  FOREIGN KEY (UserID) REFERENCES User (ID) ON UPDATE CASCADE ON DELETE RESTRICT,
  FOREIGN KEY (BookID) REFERENCES Book (ID) ON UPDATE CASCADE ON DELETE RESTRICT,
  FOREIGN KEY (AddressID) REFERENCES Address (ID) ON UPDATE CASCADE ON DELETE RESTRICT);
```

Screenshot MariaDB: Dummy-Daten in Item

```
MariaDB [bookmart]> SELECT * FROM Item LIMIT 3;
+----+--------+--------+-----------------------------------+--------------------+-----------+-----------+------------+
| ID | UserID | BookID | Quality                           | MaximumLendDuration | Shippable | AddressID | Lending    |
+----+--------+--------+-----------------------------------+--------------------+-----------+-----------+------------+
|  1 |      1 |      3 | Wie neu                           |                 21 |         1 |        10 | free       |
|  2 |      2 |      7 | Gut erhalten, leichte Gebrauchsspuren |              30 |         0 |         1 | restricted |
|  3 |      3 |      1 | Neuwertig, kaum gelesen           |                 14 |         1 |         2 | free       |
+----+--------+--------+-----------------------------------+--------------------+-----------+-----------+------------+
3 rows in set (0.000 sec)
```

- **Trigger: ON DELETE CASCADE Replikation für User**

```
CREATE OR REPLACE TRIGGER UserDeleted_Item BEFORE DELETE ON User FOR EACH ROW DELETE FROM Item WHERE UserID = OLD.ID;
```

- **Trigger: ON DELETE CASCADE Replikation für Book**

```
CREATE OR REPLACE TRIGGER BookDeleted_Item BEFORE DELETE ON Book FOR EACH ROW DELETE FROM Item WHERE BookID = OLD.ID;
```

- **Testfall: Zeige, dass Löschungen in User oder Book zur Löschung des Items führen**

```
[…] CREATE OR REPLACE PROCEDURE test_Item() […]
DECLARE uID, bID TYPE OF Item.ID;
  INSERT INTO User VALUES (NULL, 'Bsp. User', 'Name', 1, NULL, NULL, 'abcdef'); SET uID = LAST_INSERT_ID();
  INSERT INTO Item VALUES (NULL, uID, 1, NULL, 98, 0, 1, 'free');
  INSERT INTO Book VALUES (NULL, 'Bsp. Title', NULL, NULL, NULL, 1); SET bID = LAST_INSERT_ID();
  INSERT INTO Item VALUES (NULL, 1, bID, NULL, 99, 0, 1, 'free');
  DELETE FROM User WHERE ID = uID; DELETE FROM Book WHERE ID = bID;
  SELECT * FROM Item WHERE ID IN (LAST_INSERT_ID(), LAST_INSERT_ID()+1);
[…] CALL test_Item()
```

Erwartetes Testergebnis:
Empty set
Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall Item

```
MariaDB [bookmart]> CALL test_Item();
Empty set (0.010 sec)

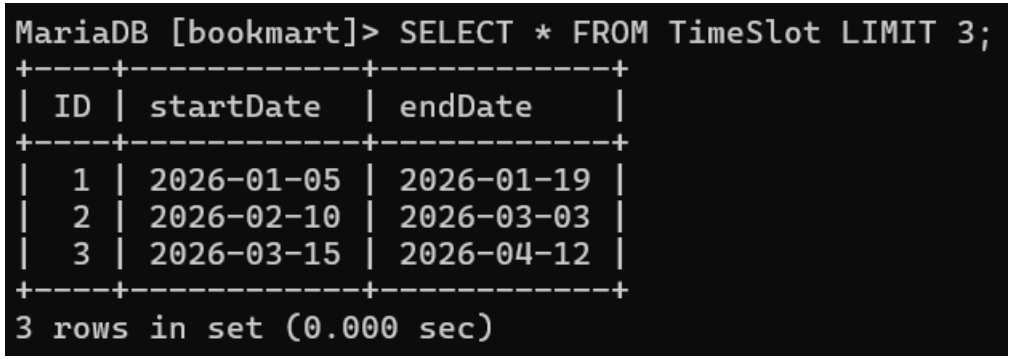
Query OK, 8 rows affected (0.010 sec)
```

TimeSlot

- **Erstellen der Tabelle**

```
CREATE OR REPLACE TABLE TimeSlot (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  startDate DATE NOT NULL,
  endDate DATE NOT NULL,
  UNIQUE INDEX (startDate, endDate),
  CONSTRAINT dates CHECK (endDate > startDate));
```

Screenshot MariaDB: Dummy-Daten in TimeSlot



```
MariaDB [bookmart]> SELECT * FROM TimeSlot LIMIT 3;
+----+------------+------------+
| ID | startDate  | endDate    |
+----+------------+------------+
|  1 | 2026-01-05 | 2026-01-19 |
|  2 | 2026-02-10 | 2026-03-03 |
|  3 | 2026-03-15 | 2026-04-12 |
+----+------------+------------+
3 rows in set (0.000 sec)
```

- **Trigger: Löscht unreferenzierte Einträge in TimeSlot**

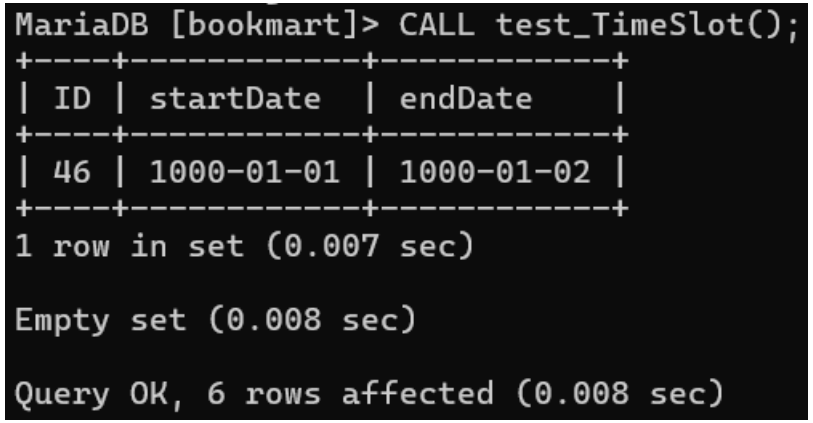
```
[…] CREATE OR REPLACE FUNCTION check_TimeSlot (tID INT UNSIGNED)
RETURNS INTEGER […] RETURN ((
    (SELECT COUNT(*) FROM Item_TimeSlot WHERE TimeSlotID = tID) + (SELECT COUNT(*) FROM Reservation_TimeSlot WHERE TimeSlotID = tID)) = 0); […]
CREATE OR REPLACE TRIGGER ItemTimeSlot_DeleteUpdate
AFTER UPDATE OR DELETE ON Item_TimeSlot FOR EACH ROW [...]
  IF check_TimeSlot(OLD.TimeSlotID) THEN DELETE FROM TimeSlot WHERE ID = OLD.TimeSlotID; […]
CREATE OR REPLACE TRIGGER ReservationTimeSlot_DeleteUpdate
AFTER UPDATE OR DELETE ON Reservation_TimeSlot FOR EACH ROW […]
  IF check_TimeSlot(OLD.TimeSlotID) THEN DELETE FROM TimeSlot WHERE ID = OLD.TimeSlotID; […]
```

- **Testfall: Zeige, dass unreferenzierte Einträge in TimeSlot ordnungsgemäß gelöscht werden**

```
[...] CREATE OR REPLACE PROCEDURE test_TimeSlot() [...]
  DECLARE tID INT UNSIGNED;
  INSERT INTO TimeSlot VALUES (NULL, '1000-01-01', '1000-01-02'); SET tID = LAST_INSERT_ID();
  INSERT INTO Item_TimeSlot VALUES (1, tID); INSERT INTO Reservation_TimeSlot VALUES (1, tID);
  DELETE FROM Item_TimeSlot WHERE TimeSlotID = tID; SELECT * FROM TimeSlot WHERE ID = tID;
  DELETE FROM Reservation_TimeSlot WHERE TimeSlotID = tID; SELECT * FROM TimeSlot WHERE ID = tID;
[…] CALL test_TimeSlot();
```

Erwartetes Testergebnis:
Erstes SELECT: Eine TimeSlot Entität wird angezeigt
Zweites SELECT: Empty set
Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall TimeSlot



```
MariaDB [bookmart]> CALL test_TimeSlot();
+----+------------+------------+
| ID | startDate  | endDate    |
+----+------------+------------+
| 46 | 1000-01-01 | 1000-01-02 |
+----+------------+------------+
1 row in set (0.007 sec)

Empty set (0.008 sec)

Query OK, 6 rows affected (0.008 sec)
```

Item_TimeSlot

- **Erstellen der Tabelle**

```
CREATE OR REPLACE TABLE Item_TimeSlot (
  ItemID INT UNSIGNED,
  TimeSlotID INT UNSIGNED,
  PRIMARY KEY (ItemID, TimeSlotID),
  FOREIGN KEY (ItemID) REFERENCES Item (ID)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  FOREIGN KEY (TimeSlotID) REFERENCES TimeSlot (ID)
    ON UPDATE CASCADE
    ON DELETE CASCADE);
```

- **Trigger: ON DELETE CASCADE Replikation für Item**

```
CREATE OR REPLACE TRIGGER ItemDeleted_ItemTimeSlot
BEFORE DELETE ON Item FOR EACH ROW
  DELETE FROM Item_TimeSlot WHERE ItemID = OLD.ID;
```

- **Trigger: Verhindere Überschneidungen der TimeSlots (Auf gleiche Weise auch für Reservation_TimeSlot implementiert)**

```
[…] CREATE OR REPLACE TRIGGER ItemTimeSlotInsert_TimeSlot
BEFORE INSERT OR UPDATE ON Item_TimeSlot FOR EACH ROW [...]
  DECLARE varItemID TYPE OF Item.ID; DECLARE varStart, varEnd TYPE OF TimeSlot.startDate; DECLARE conflicts INT;
  SELECT ID INTO varItemID FROM Item WHERE ID = NEW.ItemID;
  SELECT startDate, endDate INTO varStart, varEnd FROM TimeSlot WHERE ID = NEW.TimeSlotID;
  SELECT COUNT(*) INTO conflicts FROM Item_TimeSlot AS it RIGHT JOIN TimeSlot AS t ON it.TimeSlotID = t.ID RIGHT JOIN Item AS i ON it.ItemID = i.ID
  WHERE i.ID = varItemID AND ((varStart BETWEEN t.startDate AND t.endDate) OR (varEnd BETWEEN t.startDate AND t.endDate));
  IF (conflicts > 0) THEN SIGNAL SQLSTATE '45001' SET MYSQL_ERRNO=30002,  MESSAGE_TEXT='Tried to insert references to overlapping TimeSlots';
[...]
```

Screenshot MariaDB: Dummy-Daten in Item_TimeSlot

```
MariaDB [bookmart]> SELECT * FROM Item_TimeSlot LIMIT 3;
+--------+------------+
| ItemID | TimeSlotID |
+--------+------------+
|      3 |          1 |
|      5 |          2 |
|      1 |          3 |
+--------+------------+
3 rows in set (0.000 sec)
```

Reservation

- **Erstellen der Tabelle**

```
CREATE OR REPLACE TABLE Reservation (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  UserID INT UNSIGNED NOT NULL,
  ItemID INT UNSIGNED NOT NULL,
  FOREIGN KEY (UserID) REFERENCES User (ID)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  FOREIGN KEY (ItemID) REFERENCES Item (ID)
    ON UPDATE CASCADE
    ON DELETE RESTRICT);
```

- **Trigger: ON DELETE CASCADE Replikation für Item und User**

```
CREATE OR REPLACE TRIGGER UserDeleted_Reservation
BEFORE DELETE ON User FOR EACH ROW FOLLOWS UserDeleted_Item
  DELETE FROM Reservation WHERE UserID = OLD.ID;
CREATE OR REPLACE TRIGGER ItemDeleted_Reservation
BEFORE DELETE ON Item FOR EACH ROW FOLLOWS ItemDeleted_ItemTimeSlot
  DELETE FROM Reservation WHERE ItemID = OLD.ID;
```

Screenshot MariaDB: Dummy-Daten in Reservation

```
MariaDB [bookmart]> SELECT * FROM Reservation LIMIT 3;
+----+--------+--------+
| ID | UserID | ItemID |
+----+--------+--------+
|  1 |      1 |      1 |
|  2 |      1 |      2 |
|  3 |      2 |      3 |
+----+--------+--------+
3 rows in set (0.000 sec)
```

Reservation_TimeSlot

- **Erstellen der Tabelle**

```
CREATE OR REPLACE TABLE Reservation_TimeSlot (
  ReservationID INT UNSIGNED,
  TimeSlotID INT UNSIGNED,
  PRIMARY KEY (ReservationID, TimeSlotID),
  FOREIGN KEY (ReservationID) REFERENCES Reservation (ID)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  FOREIGN KEY (TimeSlotID) REFERENCES TimeSlot (ID)
    ON UPDATE CASCADE ON DELETE RESTRICT);
```

Screenshot MariaDB: Dummy-Daten in Reservation

```
MariaDB [bookmart]> SELECT * FROM Reservation_TimeSlot LIMIT 3;
+---------------+------------+
| ReservationID | TimeSlotID |
+---------------+------------+
|             3 |          1 |
|             7 |          1 |
|             1 |          2 |
+---------------+------------+
3 rows in set (0.000 sec)
```

- **Trigger: ON DELETE CASCADE Replikation für Reservation**

```
CREATE OR REPLACE TRIGGER ReservationDeleted_ReservationTimeSlot
BEFORE DELETE ON Reservation FOR EACH ROW
  DELETE FROM Reservation_TimeSlot WHERE ReservationID = OLD.ID;
```

- **Trigger: Verhindert Mehrfacheinträge für ein Item in einem Timeslot**

```
CREATE OR REPLACE TRIGGER ReservationTimeSlotInsert BEFORE INSERT OR UPDATE ON Reservation_TimeSlot FOR EACH ROW [...]
  IF ((SELECT ItemID FROM Reservation WHERE ID = NEW.ReservationID) IN (SELECT r.ItemID FROM Reservation_TimeSlot AS rt LEFT JOIN Reservation
  AS r ON rt.ReservationID = r.ID WHERE rt.TimeSlotID = NEW.TimeSlotID AND NOT rt.ReservationID = NEW.ReservationID)) THEN SIGNAL SQLSTATE '45000' SET
    MYSQL_ERRNO=30001, MESSAGE_TEXT='Tried inserting entries with duplicate itemIDs for specified timeslot';
```

- **Testfall: Prüfe, ob Mehrfacheinträge korrekt verhindert werden**

```
[...] CREATE OR REPLACE PROCEDURE test_ReservationTimeSlot() [...]
  DECLARE rIDa, rIDb, tID INT UNSIGNED; DECLARE errorCount TINYINT DEFAULT 0; DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    [...] ROLLBACK; RESIGNAL; END;
  [...] INSERT INTO Reservation VALUES (NULL, 1, 1), (NULL, 1, 1); SET rIDa = LAST_INSERT_ID(); SET rIDb = LAST_INSERT_ID()+1;
  DELETE FROM TimeSlot WHERE startDate = '1000-01-01'; INSERT INTO TimeSlot VALUES (NULL, '1000-01-01', '1000-01-02'); SET tID = LAST_INSERT_ID();
  INSERT INTO Reservation_TimeSlot VALUES (rIDa, tID); INSERT INTO Reservation_TimeSlot VALUES (rIDb, tID);
  SELECT * FROM Reservation_TimeSlot WHERE ReservationID IN (rIDa, rIDb);
  DELETE FROM Reservation_TimeSlot WHERE ReservationID = rIDa AND TimeSlotID = tID; DELETE FROM Handover WHERE ReservationID IN (rIDa, rIDb);
  DELETE FROM Reservation WHERE ID IN (rIDa, rIDb); DELETE FROM TimeSlot WHERE ID = tID; ROLLBACK;
[...] CALL test_ReservationTimeSlot();
```

Erwartetes Testergebnis:
ERROR 30001 (45000)
Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall Reservation_TimeSlot

```
MariaDB [bookmart]> CALL test_ReservationTimeSlot();
ERROR 30001 (45000): Tried inserting entries with du
plicate itemIDs for specified timeslot
```

Handover

Screenshot MariaDB: Dummy-Daten in Handover

```
MariaDB [bookmart]> SELECT * FROM Handover ORDER BY ReservationID DESC LIMIT 4;
+-----+---------------+------+
| ID  | ReservationID | Type |
+-----+---------------+------+
| 204 |           133 |    1 |
| 203 |           133 |    0 |
| 202 |           132 |    1 |
| 201 |           132 |    0 |
+-----+---------------+------+
4 rows in set (0.000 sec)
```

- **Erstellen der Tabelle**

```
CREATE OR REPLACE TABLE Handover (
  ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  ReservationID INT UNSIGNED NOT NULL,
  Type BOOL,
  FOREIGN KEY (ReservationID) REFERENCES Reservation (ID)
    ON UPDATE CASCADE
    ON DELETE RESTRICT);
```

Trigger: ON DELETE CASCADE Replikation für Reservation

```
CREATE OR REPLACE TRIGGER ReservationDeleted_Handover
BEFORE DELETE ON Reservation FOR EACH ROW FOLLOWS ReservationDeleted_ReservationTimeSlot
  DELETE FROM Handover WHERE ReservationID = OLD.ID;
```

- **Trigger: Erstellt zwei Handover Entitäten bei der Erstellung einer Reservation**

```
CREATE OR REPLACE TRIGGER ReservationInsert_Handover
AFTER INSERT ON Reservation FOR EACH ROW
  INSERT INTO Handover VALUES (NULL, NEW.ID, 0), (NULL, NEW.ID, 1);
```

Confirmation

Screenshot MariaDB: Dummy-Daten in Confirmation

```
MariaDB [bookmart]> SELECT * FROM Confirmation ORDER BY HandoverID DESC LIMIT 4;
+------------+--------+---------------------+
| HandoverID | UserID | conDateTime         |
+------------+--------+---------------------+
|        204 |      9 | 2027-02-28 00:00:00 |
|        204 |      6 | 2027-02-28 00:00:00 |
|        203 |      9 | 2027-02-01 00:00:00 |
|        203 |      6 | 2027-02-01 00:00:00 |
+------------+--------+---------------------+
4 rows in set (0.000 sec)
```

- **Erstellen der Tabelle**

```
CREATE TABLE Confirmation (
  HandoverID INT UNSIGNED, UserID INT UNSIGNED,
  conDateTime TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (HandoverID, UserID),
  FOREIGN KEY (UserID) REFERENCES User (ID) ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (HandoverID) REFERENCES Handover (ID) ON UPDATE CASCADE ON DELETE CASCADE);
```

- **Trigger: Stellt sicher, dass nur die jeweils beteiligten User bestätigen**

```
[…] CREATE OR REPLACE TRIGGER ConfirmationInsertUpdate_Confirmation
BEFORE INSERT OR UPDATE ON Confirmation FOR EACH ROW […]
  IF (NEW.UserID NOT IN (SELECT i.UserID AS Lender, r.UserID AS Borrower FROM Handover as h
  RIGHT JOIN Reservation AS r ON h.ReservationID = r.ID RIGHT JOIN Item AS i ON r.ItemID = i.ID
  WHERE h.ID = NEW.HandoverID)) THEN
    SIGNAL SQLSTATE '45002' SET MYSQL_ERRNO=30003,
    MESSAGE_TEXT='Tried to insert confirmation with unrelated user'; […]
```

- **Testfall: Prüfe, ob nur beteiligte User bestätigen können**

```
[…] CREATE OR REPLACE PROCEDURE test_Confirmation() […]
  DECLARE varHandoverID, varBorrower, varLender INT UNSIGNED;
  SELECT h.ID, i.UserID, r.UserID INTO varHandoverID, varLender, varBorrower FROM Handover as h
   RIGHT JOIN Reservation AS r ON h.ReservationID = r.ID RIGHT JOIN Item AS i ON r.ItemID = i.ID
   WHERE h.ID = (SELECT ID FROM Handover LIMIT 1);
  INSERT INTO Confirmation VALUES (varHandoverID,
   (SELECT ID FROM User WHERE ID NOT IN (varLender, varBorrower) LIMIT 1), NULL);
[…] CALL test_Confirmation();
```

Erwartetes Testergebnis:
ERROR 30003 (45002)
Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall Confirmation

```
MariaDB [bookmart]> CALL test_Confirmation();
ERROR 30003 (45002): Tried to insert confirmation with unrelated user
```

UserStats

- **Erstellen der Tabelle**

CREATE OR REPLACE TABLE UserStats (

UserID INT UNSIGNED PRIMARY KEY,

BooksBorrowed INT UNSIGNED DEFAULT 0,

OnTimeReturns INT UNSIGNED DEFAULT 0,

BooksLended INT UNSIGNED DEFAULT 0,

ReviewsWritten INT UNSIGNED DEFAULT 0,

FOREIGN KEY (UserID) REFERENCES User (ID) ON UPDATE CASCADE ON DELETE CASCADE);

- **Trigger: Erstellt UserStats Entitäten für neue User**

CREATE OR REPLACE TRIGGER UserInsert_UserStats AFTER INSERT ON User FOR EACH ROW

INSERT INTO UserStats (UserID) VALUES (NEW.ID);

- **Trigger: Aktualisiert BooksBorrowed, BooksLended und onTimeReturns**

[…] CREATE OR REPLACE TRIGGER ConfirmationInsert_UserStats AFTER INSERT ON Confirmation FOR EACH ROW […]

DECLARE sType, sBorrower INT UNSIGNED; DECLARE sEnd TIMESTAMP;

SELECT h.Type, r.UserID INTO sType, sBorrower FROM Handover AS h JOIN Reservation AS r ON r.ID = h.ReservationID WHERE h.ID = NEW.HandoverID;

INSERT IGNORE INTO UserStats (UserID) VALUES (NEW.UserID);

IF sType = 0 THEN IF sBorrower = NEW.UserID THEN

UPDATE UserStats SET BooksBorrowed = COALESCE(BooksBorrowed, 0)+1 WHERE UserID = NEW.UserID;

ELSE UPDATE UserStats SET BooksLended = COALESCE(BooksLended, 0)+1 WHERE UserID = NEW.UserID; […]

ELSEIF sBorrower = NEW.UserID THEN SELECT MAX(t.endDate) INTO sEnd FROM Handover AS h INNER JOIN Reservation_TimeSlot AS rt

ON rt.ReservationID = h.ReservationID INNER JOIN TimeSlot AS t ON t.ID = rt.TimeSlotID WHERE h.ID = NEW.HandoverID;

IF sEnd >= NEW.conDateTime THEN UPDATE UserStats SET OnTimeReturns = COALESCE(OnTimeReturns, 0)+1 WHERE UserID = NEW.UserID; […]

Screenshot MariaDB: Dummy-Daten in UserStats

```
MariaDB [bookmart]> SELECT * FROM UserStats ORDER BY OnTimeReturns DESC LIMIT 4;
+--------+--------------+--------------+------------+----------------+
| UserID | BooksBorrowed | OnTimeReturns | BooksLended | ReviewsWritten |
+--------+--------------+--------------+------------+----------------+
|      5 |            2 |            2 |          1 |           NULL |
|      9 |            2 |            2 |          1 |           NULL |
|      1 |            1 |            1 |       NULL |           NULL |
|      3 |         NULL |         NULL |          1 |           NULL |
+--------+--------------+--------------+------------+----------------+
4 rows in set (0.000 sec)
```

UserStats

- **Trigger: Aktualisiert ReviewsWritten**

[…] CREATE OR REPLACE TRIGGER ReviewInsert_UserStats

AFTER INSERT OR DELETE ON Review FOR EACH ROW […]

DECLARE refUserID INT UNSIGNED;

IF INSERTING THEN SET refUserID = NEW.UserID;

ELSE SET refUserID = OLD.UserID; END IF;

UPDATE UserStats SET ReviewsWritten = (SELECT COUNT(*) FROM Review WHERE UserID = refUserID)

WHERE UserID = refUserID; […]

- **Testfall: Prüfe, ob BooksBorrowed und BooksLended aktualisert werden**

[…] CREATE OR REPLACE PROCEDURE test_UserStats() […]

DECLARE lendingUserID, varItemID, varReservationID, varTimeSlotID INT UNSIGNED; START TRANSACTION;

SET lendingUserID = (SELECT UserID FROM Item WHERE NOT UserID = 1 LIMIT 1);

SET varItemID = (SELECT ID FROM Item WHERE UserID = lendingUserID LIMIT 1);

SELECT UserID, BooksLended, BooksBorrowed FROM UserStats WHERE UserID IN (1, lendingUserID);

INSERT INTO Reservation VALUES (NULL, 1, varItemID); SET varReservationID = LAST_INSERT_ID();

INSERT INTO TimeSlot VALUES (NULL, '2000-01-01', '2000-01-10'); SET varTimeSlotID = LAST_INSERT_ID();

INSERT INTO Reservation_TimeSlot VALUES (varReservationID, varTimeSlotID);

[Außerdem INSERT des Verleihers] INSERT INTO Confirmation (HandoverID, UserID, conDateTime) VALUES

((SELECT ID FROM Handover WHERE ReservationID = varReservationID AND Type = 1),

(SELECT UserID FROM Reservation WHERE ID = varReservationID),

(SELECT endDate + INTERVAL 1 DAY FROM TimeSlot WHERE ID = varTimeSlotID));

SELECT UserID, BooksLended, BooksBorrowed, OnTimeReturns FROM UserStats WHERE UserID IN (1, lendingUserID);

[Löschen der vorherigen Bestätigung und erneuter INSERT ohne “+ INTERVAL 1 DAY”] [Erneutes SELECT]

ROLLBACK; [...]

CALL test_userStats();

Erwartetes Testergebnis:

1. Select			
1	a	b	c
2	d	e	f
2. Select			
1	a	b+1	c
2	d+1	e	f
3. Select			
1	a	b+1	c+1
2	d+1	e	f

Tatsächliches Testergebnis:

Screenshot MariaDB: Testfall UserStats

```
MariaDB [bookmart]> CALL test_userStats();
+--------+-------------+---------------+---------------+
| UserID | BooksLended | BooksBorrowed | OnTimeReturns |
+--------+-------------+---------------+---------------+
|      1 |           0 |             3 |             2 |
|      2 |           2 |             0 |             0 |
+--------+-------------+---------------+---------------+
2 rows in set (0.001 sec)

+--------+-------------+---------------+---------------+
| UserID | BooksLended | BooksBorrowed | OnTimeReturns |
+--------+-------------+---------------+---------------+
|      1 |           0 |             4 |             2 |
|      2 |           3 |             0 |             0 |
+--------+-------------+---------------+---------------+
2 rows in set (0.003 sec)

+--------+-------------+---------------+---------------+
| UserID | BooksLended | BooksBorrowed | OnTimeReturns |
+--------+-------------+---------------+---------------+
|      1 |           0 |             4 |             3 |
|      2 |           3 |             0 |             0 |
+--------+-------------+---------------+---------------+
2 rows in set (0.004 sec)

Query OK, 22 rows affected (0.005 sec)
```

Zusammenfassung

Die Implementierung erfolgte auf Grundlage des zuvor erstellten Entity-Relationship-Modells und Datenwörterbuchs. Dafür wurde Tabellenweise vorgegangen.

Zunächst wurde jeweils die Tabelle erstellt und mit Dummy-Daten gefüllt.

Im Anschluss wurden Trigger erstellt, um die im ERM spezifizierten Kardinalitäten umzusetzen, die Einhaltung der fachlichen Regeln zu gewährleisten (bspw. Keine Mehrfachbelegungen eines Items für einen Timeslot) und um die Fachlogik umzusetzen (bspw. Das automatische Erhöhen von BooksLended in UserStats). Zudem wurden mehrere Trigger erstellt, welche „ON DELETE CASCADE“ replizieren, da in MariaDB die direkte Löschung einer Entität durch einen „FOREIGN KEY CONSTRAINT„ keine „ON DELETE“ Trigger auslöst.

Abschließend wurden Testfälle implementiert, meist als Prozeduren, und durchgeführt. Aufgrund der Abhängigkeiten der Tabellen, war es mehrmals erforderlich, von der genannten Reihenfolge abzuweichen und bspw. Tabellen vorab ohne Trigger anzulegen.

Während des gesamten Implementierungsprozesses, wurden die erstellten SQL-Statements parallel auf einem als Testumgebung aufgesetzten MariaDB-Server durchgeführt und in einer .sql-Datei (inkl. Kommentar) gespeichert.